

Practitioner Workbook

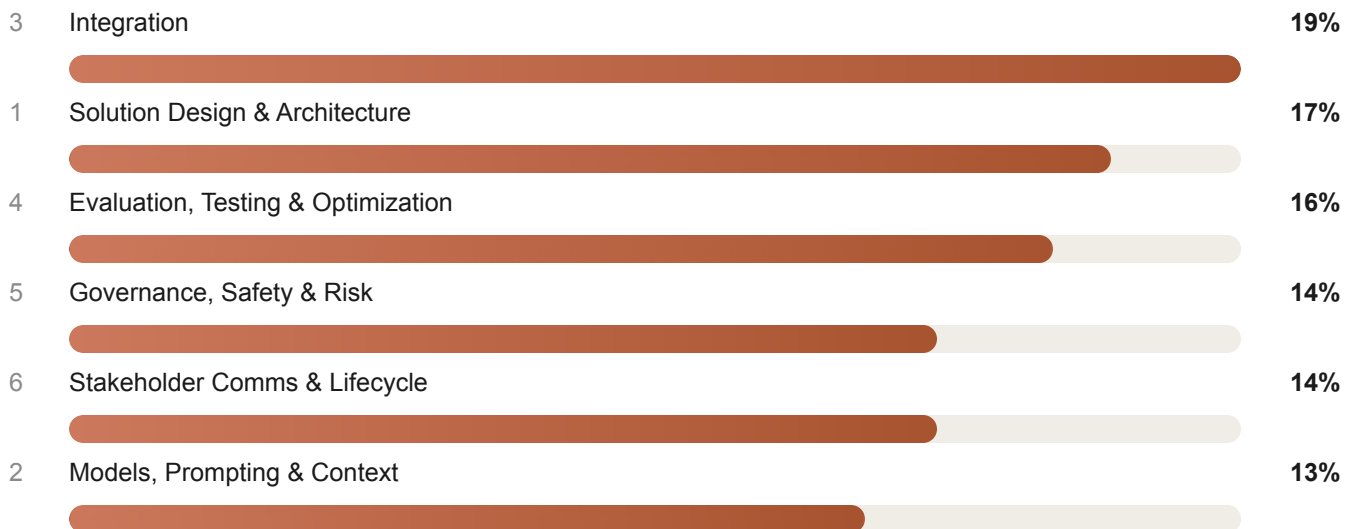
Claude Certified Architect — Professional

29 modules across 7 domains · solution architecture, RAG & integration, evaluation, governance, enterprise lifecycle

A professional-level credential for people who own the whole system: eliciting the requirement, choosing the architecture, integrating it into an enterprise safely, proving it works, governing it, and explaining the trade-off to a VP who doesn't want the jargon. This workbook is built to the published blueprint and weighted to it — **Integration is the single heaviest domain**, and the exam is scenario-based throughout.

Items	63 scored	Time	120 minutes
Passing score	720 (100–1000)	Scoring	Criterion-referenced
Fee	\$175 USD	Validity	12 months
Modules	29 · 26 design drills	Practice items	30 with rationale

Blueprint weight by domain — Integration is the heaviest; allocate study time accordingly





How to use this workbook

WHO THIS EXAM IS ACTUALLY FOR

Mid- to senior solution architects, AI/ML engineers, and technical leads. The guide recommends (but does not require) **3+ years in systems architecture or platform engineering and 6+ months hands-on with Claude or a comparable LLM system in production**. It is explicitly *not* aimed at entry-level developers or at roles limited to prompt-writing without broader system-design responsibility.

That shapes every item: you are being assessed as the person accountable for a system in an organization — not as someone who knows an API.

SOURCE & CURRENCY

Format, weights, and logistics below come from an official-format CCAR-P exam guide (v1.0, effective July 2026) surfaced via a third-party prep resource. It is more complete than the CCAR-F guide — it states full logistics. **Confirm current details with Anthropic and Pearson VUE before you register.** Platform specifics (model IDs, prices, feature names) drift; this exam tests architectural judgment, not version strings.

Logistics worth knowing cold

ITEM	DETAIL
Format	63 scored items, multiple-choice and multiple-response. Each item states how many responses to select.
Time	120 minutes — just under 2 minutes per item. Scenario stems are long; budget reading time.
Scoring	Criterion-referenced against a fixed standard from a formal standard-setting study — not curved against other candidates.
Passing	720 on a 100–1000 scale.
Results	Pass/fail plus scaled score, and percent-correct by domain. Domain percentages are informational only — pass/fail comes from the total.
Retakes	14 days after a 1st fail, 30 after a 2nd, 90 after a 3rd. Max 4 attempts per rolling 12 months; fee applies each time.
Renewal	12-month validity. On-time renewal is a free, non-proctored assessment; a lapsed credential means retaking the full exam at full fee.
Exam day	Government photo ID matching your registration exactly · stay in webcam view · clear workspace · no phones, watches, notes, or second monitors. Misconduct voids results and can revoke the credential.
Appeals	Within 14 days. The standard-setting outcome and individual item content are not appealable.

THE FIVE QUESTIONS BEHIND ALMOST EVERY CCAR-P ITEM

1. **What did the stakeholder actually ask for?** (Often the stem describes a constraint the proposed design misses.)
2. **What is the simplest architecture that satisfies it?** (Workflow before agent; agent before multi-agent.)
3. **What is the least privilege that makes it work?** (Domain 3 is the heaviest domain and this is its spine.)
4. **How would you know it works — and know when it stops working?** (Evaluation is not a launch gate; it's a lifecycle.)
5. **Who is accountable, and can you explain the trade-off to them?**

Run these five before reading the options and most distractors eliminate themselves.

HOW THE ITEMS ARE WRITTEN — AND HOW TO READ THEM

Stems are realistic vignettes with a named organization and role (a fintech loan-underwriting assistant, a healthcare clinical documentation assistant, an e-commerce returns assistant, a B2B SaaS report-generation assistant). **The domain and the vertical are mostly set dressing; the decision is generic.** Read for the constraint, not the industry. Distractors are usually real practices that are simply *the wrong control for this problem* — monitoring where prevention was needed, a confirmation prompt where scoping was needed, more model capability where more context was needed.

Contents

D1 Solution Design & Architecture — 5 modules (17%)	5
D2 Claude Models, Prompting & Context Engineering — 4 modules (13%)	10
D3 Integration — 6 modules (19%)	14
D4 Evaluation, Testing & Optimization — 4 modules (16%)	21
D5 Governance, Safety & Risk Management — 4 modules (14%)	26
D6 Stakeholder Communication & Lifecycle — 4 modules (14%)	30
D7 Developer Productivity & Enablement — 2 modules (7%)	34
— 6-week study plan, tracker & answer key	37

About 11 of 63 items. The domain that sets up every other one: pattern selection, non-functional requirements, and defensible trade-offs.

1.1 From Requirement to Architecture

~75 MIN

Objectives — Turn a business requirement into an architecture; identify the constraint that decides the design; recognise when the stated ask conflicts with the real need.

THE PROFESSIONAL-LEVEL MOVE

A CCAR-P scenario often hands you a **proposed solution** and a **constraint that quietly rules it out**. Your job is not to implement the proposal — it's to notice the conflict and say so. "The team wants an autonomous agent, and every decision must be reproducible for auditors" is not a build request; it's a mismatch you are expected to catch.

REQUIREMENT SIGNAL	WHAT IT CONSTRAINS
"reproducible", "auditable", "explain to a regulator"	Determinism. Push decisions into code; the model produces evidence, not verdicts.
"sub-second", "in the checkout flow"	Latency. Rules out agent loops and multi-hop retrieval.
"millions per month", "per-transaction margin"	Unit cost. Drives tier selection, caching, batching.
"depends what it finds", "can't enumerate the cases"	Genuine adaptivity. This is the honest agent signal.
"must never", "under no circumstances"	Enforcement. Belongs in the harness or the authorization layer, never in a prompt.
"our knowledge base changes weekly"	Freshness. Retrieval, not fine-tuned or in-prompt knowledge.

1.2 Pattern Selection

~90 MIN

Objectives — Choose among single call, workflow, agent, multi-agent, and RAG — and justify the choice against the constraint, not against sophistication.

PATTERN	FITS WHEN	CHARACTERISTIC MISUSE
Single call	One transformation: classify, extract, summarize, rewrite.	Wrapped in an agent loop that adds latency and nondeterminism for nothing.
Workflow	Known, fixed sequence with verifiable success. Extract → apply rules → output.	Replaced by an agent because "agents are more flexible" — losing the verifiability that was the requirement.
Agent	The right next step genuinely depends on what's discovered and can't be enumerated up front.	Applied to a documented procedure, producing variable output where consistency was needed.
Multi-agent	Real context isolation, parallelism, or independent verification.	Five personas sharing one context and one tool set — cost without benefit.
RAG	Answers must be grounded in a corpus that is large, changing, or authoritative.	Used where the knowledge is small and static enough to sit in a cached prompt.

THE MOST-TESTED PAIR IN THIS DOMAIN

Workflow vs. agent. Fixed sequence + easily verifiable success → **workflow**. Next step depends on what's found + cannot be enumerated → **agent**. Both directions are tested, and both wrong answers are attractive: over-engineering looks sophisticated, under-engineering looks disciplined. Read for whether the *sequence is knowable in advance* — that single property decides it.

1.3 Reference Architecture of a Claude Solution

~60 MIN

Objectives — Name the components of a production Claude system and what each owns.

LAYER	OWNS	FAILURE IF MISSING
Experience	UI, streaming, degraded states, disclosure that AI was involved.	Users see spinners and unexplained failures.
Orchestration	Pattern execution, control flow, budgets, checkpoints.	Unbounded loops; no place to enforce limits.
Prompt & context	System prompts, templates, context assembly, versioning.	Prompt drift; no reproducibility.
Knowledge	Retrieval, indexing, freshness, citations.	Stale or ungrounded answers.
Tools & integration	Capability surface, authorization, idempotency.	Over-privileged agents; duplicated side effects.
Governance	Policy, approvals, audit trail, data classification.	Nothing to show a regulator.
Evaluation & observability	Eval sets, metrics, alarms, cost tracking.	Silent regression; invisible cost growth.

USE THIS AS A GAP-FINDING CHECKLIST

When a scenario describes a design, run the seven layers and ask which one is missing. CCAR-P items very often describe an otherwise-reasonable system with exactly one absent layer — usually governance, evaluation, or authorization — and ask what you'd add before production.

1.4 Non-Functional Requirements

~60 MIN

Objectives — Elicit and design to NFRs; recognise when an NFR invalidates a pattern.

NFR	ASK THE STAKEHOLDER	ARCHITECTURAL CONSEQUENCE
Latency	p95 target? Is a human waiting?	Tight budget rules out agent loops and multi-hop retrieval; drives streaming and smaller tiers.
Throughput	Peak vs. average? Bursty?	Rate-limit headroom, queuing, batch offload.
Accuracy	What error rate is acceptable, and who bears the cost?	Sets evaluation bar, review depth, and whether automation is permissible at all.
Cost	Budget per unit of work?	Tier, caching, batching, context discipline.
Availability	What happens when the provider is degraded?	Fallback tier, cached answers, graceful unavailable state.
Compliance	Which regimes and data classes?	Residency, retention, approved channels, audit trail.

1.5 Trade-off Analysis & Decision Records

~60 MIN

Objectives — Make trade-offs explicit and document them so they survive staff turnover.

YOU CANNOT OPTIMIZE LATENCY, COST, AND ACCURACY SIMULTANEOUSLY

Name which one binds, optimize it, and **state what you gave up**. A design document claiming to improve all three at once is hiding an assumption — and on this exam, an option promising exactly that is a distractor.

THE FIVE-LINE DECISION RECORD

Context — the situation and the binding constraint · **Decision** — what you chose · **Alternatives** — what you rejected and why · **Consequences** — the cost and what it makes harder · **Revisit when** — the measurable signal that would change the decision.

That last line is what distinguishes an architect from someone with a preference, and it is the natural artifact for Domain 6's stakeholder conversations.

DESIGN DRILL 1 — FOUR SYSTEMS, FOUR ARCHITECTURES

For each, choose a pattern, name the binding NFR, and write the five-line ADR:

1. **Fintech**: loan underwriting — extract fields, apply documented policy rules, produce a decision packet. Auditors review every case.
2. **Healthcare**: clinical documentation assistant where the next step (look up, clarify, synthesize, stop) depends on what's found.
3. **E-commerce**: returns assistant handling 40,000 requests/day inside a checkout-adjacent flow.
4. **B2B SaaS**: report generation over a policy corpus that changes weekly.

Then invert each: name the constraint that would flip your pattern choice. If you can't, you chose by taste rather than by constraint — which is precisely what the exam distinguishes.

Domain 1 — scenario items

ITEM P1-1 · MODULE 1.2

A fintech platform team needs a loan-underwriting assistant to follow the same known sequence every time — extract fields, apply policy rules, produce output — with easily verifiable success. Which pattern fits?

- A. An autonomous agent that selects its own next step
- B. A workflow-based pattern with fixed, verifiable steps
- C. A multi-agent system with a specialist per step
- D. A single prompt performing all three stages

B. Known sequence plus easily verifiable success is the workflow signature. A and C add variability and complexity the task doesn't need — and in underwriting, variability is a regulatory problem, not just an engineering one. D collapses three independently verifiable steps into one opaque one, discarding the verifiability that was a stated requirement.

ITEM P1-2 · MODULE 1.2

A healthcare team is designing a clinical documentation assistant where the right next step — look something up, ask a clarifying question, synthesize, or stop — genuinely depends on what's discovered along the way. Which architecture fits?

- A. A fixed workflow with branches for each known case
- B. An agentic architecture that decides the next step from what it finds
- C. A single call with a long system prompt
- D. A batch pipeline

B. This is the honest agent signal: the path is data-dependent and cannot be enumerated in advance. A is the mirror-image error of item P1-1 — a fixed workflow cannot adapt to genuinely variable paths, and the branch thicket grows without ever covering the space. Note that the clinical setting still demands human review of output; adaptivity and oversight are separate decisions.

ITEM P1-3 · MODULE 1.1

A team proposes an autonomous agent for a process where every decision must be reproducible and explainable to an auditor. As the architect, what's your response?

- A. Build the agent as requested; add logging to satisfy auditors
- B. Flag the conflict: model-driven control flow undermines reproducibility — move decisions into code and use Claude for the language-understanding steps
- C. Build the agent with a more capable model for consistency
- D. Build the agent and have auditors review a sample

B. The stated requirement and the proposed design are incompatible, and surfacing that is the architect's job. A is the trap: logs record *what* happened but don't make the process reproducible, and auditors asking "why this decision" won't be satisfied by a transcript. C misreads reproducibility as capability. D samples a process that remains non-reproducible case to case.

ITEM P1-4 · MODULE 1.4

A returns-processing assistant must respond within a strict p95 latency budget inside a customer-facing flow. The proposed design uses a multi-step agent with retrieval at each step. What's the concern?

- A. Cost will be higher than a single call
- B. The pattern is incompatible with the latency NFR — each agent iteration and retrieval hop adds serial round trips
- C. Retrieval quality may be poor
- D. The agent may need more tools than planned

B. A latency NFR is a hard architectural constraint, and multi-step agent loops are serially additive — you cannot tune your way under the budget. A is true but secondary; the design fails the requirement outright regardless of cost. C and D are implementation risks, not the mismatch between the chosen pattern and a stated non-functional requirement.

D2 Claude Models, Prompting & Context Engineering

13% · 4 modules

About 8 of 63 items. Tested as architectural choices — where instructions live, how context is assembled, how a model is chosen — not as prompt-writing craft.

2.1 Model Selection as a Multi-Factor Trade-off

~60 MIN

Objectives — Select a model against several dimensions at once; defend the choice against both "always biggest" and "always cheapest."

THE FRAMING THIS DOMAIN TESTS

Model selection is a **genuine multi-factor trade-off** — reasoning ability, response speed, context capacity, and cost — matched to a specific workload's complexity, quality bar, latency target, and budget. Any option that optimizes a single factor in isolation is wrong, and that includes the flattering ones ("use the most capable model to be safe").

TIER	PROFILE	ARCHITECTURAL FIT
Haiku	Fastest, cheapest, lightest reasoning	High-volume, latency-critical, well-specified: classification, routing, extraction, triage.
Sonnet	Balanced	The production default for most knowledge work and coding.
Opus	Deepest reasoning, highest cost/latency	Hard, high-stakes reasoning; long-horizon agentic work.

SELECTION IS PER-STEP, NOT PER-SYSTEM

In a pipeline or multi-agent design each step gets its own decision — a triage classifier and a final synthesis step have different requirements. **Tiered routing** (cheap tier triages, capable tier handles the flagged minority) is the standard answer when volume and difficulty conflict, and it appears regularly as the correct option.

2.2 Where Instructions Live

~60 MIN

Objectives — Place persistent behavior, per-request detail, and hard constraints at the right layer.

LAYER	NATURE	BELONGS HERE
Organizational policy / harness controls	Enforced	Absolute constraints. Anything phrased "must never."
System prompt	Advisory	Persona, standing behavioral guardrails, output conventions, escalation rules — applied on every call regardless of the request.
Templates / assembled context	Advisory	Reusable structures, retrieved material, few-shot examples.
User message	Advisory	The specific request and its data.

THE DISTINCTION THE ITEMS TURN ON

Persistent, request-independent behavior → **the system prompt**. Repeating a persona in every user message is inconsistent and unmaintainable. But a system prompt is still *advisory* — the model weighs it. A requirement that must hold even under prompt injection belongs in an **enforced** layer: authorization, harness controls, or approval workflow. Both halves of that get tested.

2.3 Context Engineering

~75 MIN

Objectives — Treat context as a managed budget; choose the right mechanism for the symptom.

MECHANISM	WHAT IT DOES	CHOOSE WHEN
Context editing	Clears stale tool results — prunes, doesn't summarize.	Agent-loop bulk is old tool output.
Compaction	Summarizes earlier context.	Long dialogue whose gist must survive.
Memory / external store	Durable state.	Anything that must persist across sessions — the only mechanism that does.
Retrieval	Injects relevant slices on demand.	Corpus larger than any window, or changing independently.
Caching	Reuses a stable prefix.	The same long prefix on repeated calls — a cost lever with a design constraint attached.

DIAGNOSE CONTEXT BEFORE CAPABILITY

A system that starts well and **degrades over the course of a run** — contradicting itself, repeating work, losing the thread — has a context problem, not a model problem. The instinct to upgrade the model is expensive and doesn't fix it. Quality that is poor *from the first turn* is a different diagnosis.

2.4 Prompt Lifecycle

~45 MIN

Objectives — Manage prompts as versioned artifacts across a system's life.

- 1 **Version prompts with the code that uses them** — a prompt change is a behavior change and deserves the same review.

- 2 **Test against an eval set before rollout** (Domain 4). A prompt improvement validated on one example is not validated.
- 3 **Keep prompt, schema, and eval set in sync** — changing one usually requires changing all three.
- 4 **Centralize shared prompts** so behavior is consistent across teams and one fix propagates.
- 5 **Watch prefix stability** — edits to a cached prefix have a cost consequence, not just a behavioral one.

Domain 2 — scenario items

ITEM P2-1 · MODULE 2.2

A fintech team wants a loan-underwriting assistant to follow a consistent persona and standing behavioral guardrails on every call, regardless of the specific user request. Where should this be configured?

- A. Repeated at the top of every user message
- B. In the system prompt, the dedicated mechanism for persistent cross-call instructions
- C. In the tool descriptions
- D. In a retrieval corpus the model queries when unsure

B. System prompts exist for exactly this: persistent instructions, templates, and guardrails applying across every call, distinct from per-request content. A works but is fragile, inconsistent, and unmaintainable across a team. C misuses a field meant to describe capabilities. D makes standing behavior conditional on the model choosing to look it up.

ITEM P2-2 · MODULE 2.1

A B2B SaaS team must choose a model for a report-generation assistant, weighing reasoning ability, speed, context capacity, and cost together. What does this reflect?

- A. That the most capable model is always correct for production
- B. That model selection is a multi-factor trade-off matched to the workload's complexity, quality bar, latency target, and budget
- C. That the cheapest model should be chosen and escalated only on complaints
- D. That model choice matters less than prompt quality

B. The dimensions are weighed *together* against a specific workload. A and C are the two single-factor failure modes the exam consistently rejects — one wastes cost and latency, the other discovers quality problems through customer complaints. D is false: tier selection is typically the largest single cost and latency lever available.

ITEM P2-3 · MODULE 2.3

A long-running assistant produces good answers early in a session but by later turns contradicts its own earlier decisions. What's the most likely architectural cause?

- A. The model tier is too low
- B. Context growth is unmanaged — history and tool results crowd the window and earlier decisions lose salience
- C. Temperature is too high
- D. The system prompt is too short

B. Degradation *over the course of a run* is the signature of context exhaustion. A is the expensive reflex that leaves the cause untouched — the same degradation reappears a few turns later. The fix is a context strategy: editing, compaction, or externalizing decisions into durable state.

D3 Integration

19% · 6 modules — heaviest domain

About 12 of 63 items — the single heaviest domain. Its spine is **least privilege**: most items ask what an integration should be allowed to do, not how to call it.

3.1 Integration Topology

~60 MIN

Objectives — Choose synchronous, asynchronous, batch, or event-driven integration from the workload's latency and volume profile.

MODE	FITS	DESIGN IMPLICATIONS
Synchronous	A user is waiting.	Latency budget, streaming, timeout and degraded-response design.
Asynchronous / queued	Work is submitted and collected later.	Durable queue, status surface, idempotent processing, retry policy.
Batch	Nobody waiting on any single result — overnight enrichment, bulk classification.	Substantially cheaper per token; results keyed by request id, not order.
Event-driven	React to changes in a system of record.	Deduplication, ordering, poison-message handling, backpressure.

3.2 RAG Architecture

~90 MIN

Objectives — Design a retrieval pipeline and know where each stage fails.

STAGE	DECISION	FAILURE IT CAUSES WHEN WRONG
Ingestion	Sources, refresh cadence, change detection.	Stale answers that look current — the most damaging RAG failure because nothing signals it.
Chunking	Size and boundaries.	Chunks that split a rule from its exception; retrieved fragments that are individually true and jointly misleading.
Retrieval	Ranking, top-k, filters, permissions.	Missing the passage that mattered; or returning content the user isn't entitled to see.
Assembly	Ordering, delimiting, budget.	Context bloat; untrusted content read as instruction.
Generation	Grounding instructions, citations, refusal path.	Ungrounded claims presented with retrieved-looking authority.
Verification	Citation checks, coverage, "not found" handling.	Fabrication when the corpus lacks the answer.

TWO RAG RULES THAT CARRY MOST OF THE EXAM WEIGHT

- 1. Retrieval must respect the user's permissions.** A retrieval layer that ignores per-user entitlements turns your assistant into a data-exfiltration path — the model will faithfully summarize whatever it was handed. Filter at retrieval time, by the requesting user's identity.
- 2. Retrieved content is untrusted data, never instructions.** External or user-supplied content in the context can attempt to steer the model (Domain 5). Delimit it, say so explicitly, and restrict what sensitive actions it can trigger without review.

GIVE THE SYSTEM A WAY TO SAY "NOT IN THE CORPUS"

If the design has no explicit not-found path, a retrieval miss becomes a fabrication — the model must produce *something*. An explicit refusal-or-escalate branch is a design requirement, not a nicety, and it is the difference between a system that fails visibly and one that fails silently.

3.3 Tools, MCP & Enterprise Connectivity

~75 MIN

Objectives — Design the capability surface; choose MCP where standardization pays.

THE MCP ARGUMENT, STATED THE WAY THE EXAM WANTS IT

MCP is an open protocol exposing tools and data through standardized servers any compliant client can consume. Without it, every (client × system) pair needs bespoke integration — **N×M**. With it, each system is exposed once and each client consumes it — **N+M**. That combinatorial reduction is the architectural justification; accuracy and caching are not.

CAPABILITY DECISION	GUIDANCE
Broad tool vs. dedicated tools	Promote an action to its own tool when it must be gated, audited, rendered, or parallelized . A broad exec tool gives your harness an opaque string it cannot govern.
Tool count	Keep the surface focused; schemas consume context and dilute selection. Use tool search when the library must be large.
Tool descriptions	State when to call, not just what it does — under-triggering is usually a description problem.
Credentials	Never in model-readable context . Host-side, or vault-injected after the request leaves the model's reach.

3.4 Identity, Authorization & Least Privilege

~90 MIN

Objectives — Scope capability to the role and the request — the single most tested idea in the heaviest domain.

LEAST PRIVILEGE, PRECISELY

Remove capabilities the role does not require — eliminating attack surface rather than merely watching it. Logging and confirmation prompts are *detective* and *compensating* controls; they observe or interrupt misuse, they do not prevent it. When an item offers "log it", "alert on it", or "ask the user to confirm" against "don't grant the capability," the answer is not granting it.

ANTI-PATTERN	WHY IT FAILS	CORRECT DESIGN
One shared credential with full account access, used identically for every request	Violates least privilege and maximizes blast radius on any compromise or bug.	Scope authorization to what each request and role actually needs.
Agent holds every capability the platform offers	Support staff who only read records and draft replies don't need refund and delete.	Remove the unneeded tools entirely from that agent's surface.
Acting with the service's identity, not the user's	Bypasses per-user entitlements; every user sees everything the service can.	Propagate user identity; authorize per request.
Confirmation prompt as the only guard on a destructive tool	Compensating control; fatigue defeats it, and it can be socially engineered.	Don't grant it — or gate it in the authorization layer.

3.5 Data Flow & Boundaries

~60 MIN

Objectives — Decide what data crosses which boundary, and prove it.

- 1 **Classify before you design.** What classes exist (PII, PHI, financial, confidential), and which may enter a prompt at all?
- 2 **Minimize at the boundary.** Send the fields the task needs — a returns assistant rarely needs a full customer record to draft a reply.
- 3 **Redact or tokenize** what must cross but needn't be legible.
- 4 **Know retention and residency** for each hop, including your own logs — which are frequently the overlooked copy.
- 5 **Keep an audit trail** sufficient to reconstruct what was sent, retrieved, and returned.

3.6 Integration Resilience

~60 MIN

Objectives — Design for the failure of every dependency.

CONDITION	DESIGNED RESPONSE
Rate limited	Exponential backoff with jitter; queue if latency-tolerant, shed with a clear message if not.
Provider degraded	Retry, then fall back — smaller tier, cached answer, or an explicit unavailable state.
Downstream tool down	Degrade to the capability that still works and say which part is unavailable .
Retry after an unclear outcome	Idempotency. A retried refund must not issue twice.
Malformed model output	Validate, then reject / retry / escalate — never silently drop (Domain 4).

DESIGN DRILL 3 — AUTHORIZATION REVIEW

Take the account-support assistant: it can read records, draft replies, issue refunds, and delete accounts. The support staff using it only ever read records and draft replies.

- Redesign the capability surface. What is removed, what is gated, what remains?
- Whose identity does each call act under, and where is that enforced?
- A refund tool is genuinely needed by a *different* role — how do you serve both without one over-privileged agent?
- Write the five-line ADR, and make "Revisit when" concrete.

Domain 3 — scenario items

ITEM P3-1 · MODULE 3.4

An account-support agent can read records, draft replies, issue refunds, and delete accounts, but the support staff using it only ever need to read records and draft replies. Applying least privilege, what should change?

- A. Keep all capabilities but log every refund and deletion
- B. Keep all capabilities but require a confirmation prompt for refunds and deletions
- C. Remove the refund and delete capabilities from this agent's surface entirely
- D. Keep all capabilities but alert a manager when they're used

C. Least privilege means *removing* capabilities the role doesn't require — eliminating the attack surface rather than observing it. A and D are detective controls that tell you afterwards; B is a compensating control defeated by fatigue or a convincing prompt. All three leave a capability present that nobody in this role needs, which is the definition of unnecessary privilege.

ITEM P3-2 · MODULE 3.4

A team configured a customer-support agent with a single shared API credential granting full account access, used identically for every request regardless of need. What's the primary architectural problem?

- A. Credential rotation becomes harder
- B. It violates least privilege and expands the blast radius of any compromise or bug
- C. It makes per-user analytics harder
- D. Shared credentials are slower
- E. It complicates local development

B. A single broad credential used for everything means any compromise — or any bug, or any successful prompt injection — reaches the full scope of that credential. Authorization should be scoped to what each request actually needs. A and C are genuine secondary costs; D and E are minor operational annoyances. Only B names the security property being violated.

ITEM P3-3 · MODULE 3.2

A RAG assistant serves employees with differing entitlements. It retrieves from a shared index using a service identity. What must change before production?

- A. Increase top-k so relevant results aren't missed
- B. Filter retrieval by the requesting user's entitlements so the model is never handed content that user may not see
- C. Instruct the model not to reveal restricted content
- D. Add citations so users can verify sources

B. Authorization belongs at retrieval, enforced by identity — once restricted content is in the context, the model will faithfully use it, and you have built an exfiltration path. C attempts to solve an access-control problem with an advisory instruction the model merely weighs. D is good practice that would, here, helpfully cite documents the user was never entitled to see. A makes the exposure larger.

ITEM P3-4 · MODULE 3.6

A returns assistant issues refunds through a downstream payments API. A network timeout leaves the outcome unknown. What must the integration guarantee?

- A. That it never retries after a timeout
- B. That refund operations are idempotent, so a retry cannot issue a second refund
- C. That timeouts are logged for reconciliation
- D. That the model re-checks whether the refund succeeded

B. The failure mode is a successful server-side operation whose response was lost — retrying is correct, duplicating a refund is not, and idempotency keys resolve exactly this. A abandons resilience to avoid the problem. C detects it after money moved. D asks the model to adjudicate a financial transaction, which is neither reliable nor the model's responsibility.

ITEM P3-5 · MODULE 3.1

A B2B SaaS platform must enrich 2 million stored records with model-generated summaries. Nothing is user-facing and the work must finish within 24 hours. Best integration mode?

- A. Synchronous calls from the web tier
- B. Batch processing, with results collected by request identifier
- C. Event-driven processing triggered per record write
- D. A long-running agent iterating over the dataset

B. High volume with no one waiting is the batch profile, at substantially lower cost per token. A puts a bulk job on the latency-critical serving path. C suits ongoing change capture, not a one-time backfill of existing records. D applies an agent loop to a uniform, well-specified transformation — maximum cost and nondeterminism for no benefit.

D4 Evaluation, Testing & Optimization

16% · 4 modules

About 10 of 63 items. The recurring theme: evaluation is a strategy spanning several dimensions and a lifecycle — not a single metric checked once before launch.

4.1 Evaluation Strategy & Metrics

~75 MIN

Objectives — Define a metric set proportionate to the stakes; reject single-metric evaluation.

WHAT "COMPLETE" MEANS HERE

A complete evaluation strategy defines metrics across **accuracy, latency, cost, safety, and security**, as appropriate to the feature's stakes — not one metric in isolation. Any option offering a single number (however good that number is) as the evaluation strategy is wrong.

DIMENSION	REPRESENTATIVE MEASURES	MISSING IT MEANS
Accuracy	Task success rate, groundedness, citation validity, error rate by type.	You ship something confident and wrong.
Latency	p50 / p95 end-to-end; time to first token for streamed UX.	Passes offline, unusable in the flow. Agent latency is long-tailed — the mean hides it.
Cost	Cost per request and per resolved task; token split by cache read/write.	Quality at an indefensible unit economic.
Safety	Harmful-output rate, refusal appropriateness, injection resistance.	Reputational and regulatory exposure.
Security	Privilege escalation attempts, data-leak checks, entitlement bypass.	An assistant that becomes an exfiltration path.

MEASURE PER RESOLVED TASK, NOT PER CALL

A cheaper model that needs three attempts is not cheaper. A faster model that escalates half its cases to humans is not faster. **Evaluate at the unit the business cares about** — a resolved ticket, an approved document, a completed underwriting decision — which is also the unit stakeholders in Domain 6 will ask about.

4.2 Evaluation Datasets & Methodology

~75 MIN

Objectives — Build a dataset that catches real failures; combine automated and human grading appropriately.

THE DATASET RULE

A representative evaluation dataset includes **edge cases and ambiguous scenarios** — not just easy, typical cases — and is assessed with a **mix of automated and human methodologies**. A dataset of happy-path examples produces a passing score and no information: it cannot fail, so it cannot warn you.

MUST INCLUDE	WHY
Typical cases	Baseline competence and regression detection.
Edge cases	Rare inputs where systems actually break in production.
Ambiguous cases	Reveals whether the system asks, escalates, or guesses — often the highest-value signal in the whole set.
Adversarial cases	Injection attempts, entitlement probes, out-of-scope requests.
Known past failures	Every production incident becomes a permanent regression test.
Negative cases	Inputs where the correct answer is "not in the corpus" or a refusal.

GRADING METHOD	BEST FOR	CAUTION
Exact / structural match	Classification, extraction, schema conformance.	Cheapest and most reliable — use wherever the task allows.
Programmatic checks	Compiles, tests pass, citations resolve, totals reconcile.	Objective and underused.
LLM-as-judge	Open-ended quality with no single right answer.	Needs an explicit rubric, a separate context, and periodic calibration against human ratings.
Human review	Ground truth for consequential output.	Expensive — spend it calibrating the cheaper methods.

4.3 Testing Across the Lifecycle

~60 MIN

Objectives — Place evaluation gates from development through production.

- 1 **Development** — run the eval set on every prompt, schema, tool, or model change. Non-determinism means a single passing run proves nothing.
- 2 **Pre-release** — regression against known failures; adversarial pass; cost and latency profiling at expected volume.
- 3 **Release** — canary or staged rollout with automatic rollback criteria defined *before* you ship.
- 4 **Production** — continuous sampling and human spot-checks; alarms on the metrics from 4.1.
- 5 **Post-incident** — every incident becomes a permanent eval case.

MODEL AND PLATFORM CHANGES ARE CHANGE EVENTS

Behavior can shift when you move models, adjust effort, or change a prompt. **Re-run the eval set** — a version change is not a no-op, and "we only changed the model" is how silent regressions ship. This also means your eval set is infrastructure: version it, own it, and keep it current with the product.

4.4 Optimization

~60 MIN

Objectives — Improve cost, latency, or quality without trading away the others unknowingly.

GOAL	LEVERS, ROUGHLY IN ORDER	WATCH
Cost	Right-size the tier per step · batch latency-tolerant work · cache the stable prefix · control context growth · tune effort.	Re-run evals — every one of these can move quality.
Latency	Smaller tier · stream · parallelize independent work · shorten retrieval hops · trim context.	Streaming improves <i>perceived</i> latency, not total time.
Quality	Better context and grounding · schema enforcement · few-shot examples · escalate tier · add verification.	Measure aggregate movement, not one improved example.

DESIGN DRILL 4 — AN EVALUATION STRATEGY FOR A HIGH-STAKES FEATURE

For the fintech loan-underwriting assistant, specify: the metric set across all five dimensions with a target for each · the dataset composition (how many typical, edge, ambiguous, adversarial, negative) · the grading method per metric · the gates at each lifecycle stage · the rollback criteria · and the two alarms you'd wire first. **Then:** state what your evaluation would *not* catch. Every strategy has a blind spot, and knowing yours is the professional-level answer.

Domain 4 — scenario items

ITEM P4-1 · MODULE 4.1

A B2B SaaS team is defining evaluation metrics for a report-generation assistant — a high-stakes feature reviewed by the VP of Engineering. Which set best reflects a complete strategy?

- A. Accuracy alone, since correctness is what matters
- B. Accuracy, latency, cost, safety, and security, scaled to the feature's stakes
- C. User satisfaction survey scores
- D. Token consumption and request volume

B. A complete strategy spans the dimensions that determine whether the feature can actually run in production. A is the most tempting wrong answer — correctness matters most, but a perfectly accurate feature that is too slow, too expensive, or leaks data still cannot ship. C is lagging and subjective; D measures activity, not value.

ITEM P4-2 · MODULE 4.2

A fintech team is building an evaluation dataset for a loan-underwriting assistant and wants it to catch real failure modes before launch. What should it include?

- A. A large sample of typical, representative applications
- B. Edge cases and ambiguous scenarios alongside typical ones, assessed with a mix of automated and human methods
- C. Synthetic applications generated by the model itself
- D. The most recent 100 applications

B. Systems fail at the edges and on ambiguity, so a dataset without them cannot surface the failures you're trying to prevent — and a set that always passes gives no information. A alone measures the easy path. C risks inheriting the model's own blind spots as your ground truth. D is a convenience sample with no guarantee of covering hard cases.

ITEM P4-3 · MODULE 4.3

A team plans to switch the underlying model to reduce cost. Their feature has a mature eval set. What should the rollout include?

- A. Switch directly — the eval set was validated on the previous model
- B. Re-run the full eval set on the new model, compare across all metric dimensions, and stage the rollout with defined rollback criteria
- C. Switch and monitor customer complaints
- D. Switch only for new customers

B. A model change is a behavior change: quality, latency, and cost can all move, sometimes in opposite directions. A treats a prior validation as transferable, which is exactly backwards. C discovers regressions through customers — the most expensive detector available. D limits blast radius but still ships an unevaluated change, and now you're running two behaviors with no comparison data.

ITEM P4-4 · MODULE 4.1

A cheaper model reduces per-call cost by 40%, but resolution now takes an average of 2.3 attempts instead of 1.1. How should the architect evaluate this?

- A. Adopt it — per-call cost is the relevant measure
- B. Measure cost per *resolved task*, including retries and any escalation, before deciding
- C. Reject it — cheaper models are false economy
- D. Adopt it and add capacity to absorb the retries

B. Per-call cost is the wrong unit when attempts change: $2.1 \times$ the attempts at $0.6 \times$ the price is a net increase, before counting added latency and any human escalation. A optimizes the measurable rather than the meaningful. C is a rule of thumb, not an analysis — cheaper tiers often win on the right workload. D accepts the economics without ever computing them.

About 9 of 63 items. Assessed as design responsibility: what you document, what you prevent, and who is accountable.

5.1 Documented Limitations & Risk Identification

~60 MIN

Objectives — Name the genuine, well-documented limitations of LLM systems and design around them rather than assuming configuration removes them.

LIMITATION	WHAT IT ACTUALLY MEANS	DESIGN RESPONSE
Hallucination	Fluent, confident output not grounded in any source. Not eliminated by configuration.	Grounding and retrieval · citation checks · verification before action · human review where consequential.
Training cutoff	No knowledge of events after training.	Retrieval or tools for anything current.
No inherent system access	Knows nothing about your data unless supplied.	Explicit integration; never assume awareness.
Non-determinism	Same input can yield different output.	Eval sets over aggregates · determinism in code where reproducibility is required.
Prompt injection susceptibility	Content in context can attempt to steer behavior.	Treat retrieved/user content as data · least privilege · gate sensitive actions.
Context limits	Finite window; quality degrades as it fills.	Context strategy (Domain 2.3).

THE FRAMING THE EXAM REWARDS

These are **documented properties to design around**, not defects to be configured away. An option claiming a limitation is "solved" by a setting, a bigger model, or a stern instruction is wrong. Production designs *acknowledge* the limitation and add a compensating control.

5.2 Safety Guardrails & Prompt Injection

~75 MIN

Objectives — Design layered guardrails, especially where untrusted content enters via RAG.

THE RAG INJECTION ANSWER, STATED PRECISELY

Prompt injection from untrusted retrieved content is mitigated by **treating that content strictly as data, never as instructions**, and by **restricting what sensitive actions it can trigger without review**. It is *not* mitigated by trusting the source, by a larger model, or by phrase detection alone.

LAYER	TYPE	CONTRIBUTION
Delimit untrusted content; instruct it is data	Preventive	Necessary, not sufficient — it is still text competing with text.
Least privilege on tools	Preventive	Strongest single control: a successful injection has nothing to hijack.
Approval on sensitive actions	Preventive	Injection cannot complete an irreversible act alone.
Credentials outside model context	Preventive	Nothing to exfiltrate even under a successful injection.
Output filtering / egress checks	Detective	Catches some exfiltration attempts in flight.
Monitoring and audit	Detective	Tells you afterwards — necessary, never primary.

5.3 Privacy, Compliance & Data Governance

~60 MIN

Objectives — Apply classification, minimization, and regime requirements to a design.

- 1 **Classify** the data classes in play — PII, PHI, financial, confidential, special category.
- 2 **Minimize** what enters a prompt; send fields the task requires, redact or tokenize the rest.
- 3 **Map the regime** — GDPR, HIPAA, sector rules, and your **internal policy**, which is often stricter than law and most often overlooked.
- 4 **Control retention** across every hop, including logs and eval datasets built from production traffic.
- 5 **Preserve rights** — access, correction, deletion — including data captured in memory stores or eval sets.
- 6 **Escalate** to privacy, security, or legal when policy is silent. **Silence is not permission.**

5.4 Human Oversight & Accountability

~60 MIN

Objectives — Decide where humans must remain in the loop; make oversight real.

MEANINGFUL OVERSIGHT REQUIRES FOUR THINGS

A named human who is **competent** to evaluate the output, has the **information and time** to do it, holds **genuine authority to reject**, and is **accountable** for the result. Missing any one makes the control a formality. A near-100% approval rate at high volume is the diagnostic signature of a gate that has stopped functioning.

WHERE A HUMAN MUST DECIDE

Consequential decisions about people (credit, employment, benefits, clinical care), regulated professional advice, irreversible financial action, and anything a regulator will ask you to justify. Claude drafts, summarizes, and surfaces options; a qualified human decides and owns the outcome. "The system decided" is not an answer to a regulator — and "the model was confident" is not an oversight mechanism.

Domain 5 — scenario items

ITEM P5-1 · MODULE 5.2

A healthcare team builds a RAG feature for a clinical documentation assistant that ingests content from external, untrusted sources. Which guardrail most directly addresses prompt injection from that content?

- A. Trust content only from sources on an approved vendor list
- B. Treat retrieved content strictly as data, never as instructions, and restrict which sensitive actions it can trigger without review
- C. Use a more capable model less susceptible to manipulation
- D. Scan retrieved documents for known injection phrases

B. This pairs the preventive control (content is data, not instruction) with the containment control (limit what it can trigger unreviewed), addressing the class rather than one phrasing. A misplaces trust — an approved vendor's document can still be compromised or contain user-generated text. C treats a systems problem as a capability problem. D is an unbounded blocklist against an attacker who can rephrase, translate, or encode.

ITEM P5-2 · MODULE 5.1

An e-commerce team documents known risks for a returns-processing assistant before deployment. Which is a genuine, well-documented limitation the design must account for?

- A. Hallucination — fluent output not grounded in any source, addressed through verification, grounding, or review
- B. Claude cannot process structured data
- C. Claude cannot follow multi-step instructions
- D. Hallucination, which is eliminated by correct configuration

A. Hallucination is a real, documented property requiring compensating controls in the design. **D is the trap** — it names the right limitation and then makes a false claim about it; no configuration, model choice, or instruction eliminates it, which is precisely why the design must account for it. B and C are simply untrue.

ITEM P5-3 · MODULE 5.4

A lending platform proposes that the assistant issue final approve/decline decisions automatically for applications it scores as unambiguous. What's the governance assessment?

- A. Acceptable — unambiguous cases carry little risk
- B. Acceptable if the threshold is tuned conservatively
- C. Inappropriate — credit decisions materially affect people, are regulated, and require an accountable human decision-maker
- D. Acceptable if applicants are told AI was involved

C. Credit is a regulated, consequential decision about a person, carrying fair-lending and adverse-action obligations — it needs an accountable human, not a confidence threshold. A and B both rest on the model's own assessment of ambiguity, which fails exactly where it matters. D is a genuine transparency obligation that does not change who is accountable for the decision.

ITEM P5-4 · MODULE 5.3

A team wants to build an evaluation dataset from production transcripts containing customer PII. What should the architect require?

- A. Nothing — it's internal use of data you already hold
- B. Classification, minimization or redaction, a retention policy, and confirmation that this use is permitted for that data class
- C. Approval from the engineering manager
- D. Only that the dataset be stored encrypted

B. An eval set built from production traffic is a new copy of personal data with its own purpose, retention, and rights obligations — it is exactly the kind of secondary use that bypasses review because it feels like engineering hygiene. A assumes collection permits any downstream use. C is the wrong approver for a privacy decision. D is one control among several and doesn't address lawful basis or retention.

About 9 of 63 items. The domain engineers most often underestimate — and the one that most clearly separates a professional architect from a senior developer.

6.1 Structured Discovery

~75 MIN

Objectives — Run discovery before design; surface constraints, outcomes, and risks while they're still cheap to address.

WHY DISCOVERY COMES FIRST

Structured discovery conducted **before** design exists to surface hard operational constraints, expected outcomes, and risks. Skipping it — or doing it after a design exists — is precisely how a binding constraint is discovered late, when rework is expensive and commitments have been made. On this exam, "we started designing and later found out..." is always a discovery failure, and the fix is never "add it now and move on."

DISCOVERY AREA	QUESTIONS THAT CHANGE THE ARCHITECTURE
Outcome	What does success look like in business terms? How will you know in 90 days?
Operational constraints	Peak volume? Latency? Hours of operation? Existing SLAs this must not break?
Data	Which systems, which classes, which entitlements, which residency rules?
Process	Who does this today? What happens to their work? Who approves exceptions now?
Risk	What's the cost of a wrong answer, and who bears it?
Compliance	Which regimes? Who signs off? What must be auditable?
Adoption	Who has to change behavior, and what would make them refuse?

6.2 Communicating Trade-offs

~75 MIN

Objectives — Translate a technical trade-off into the terms the audience owns.

THE COMMUNICATION RULE

Effective architecture communication **translates technical trade-offs into terms the audience cares about** — user experience, budget, SLA impact, risk exposure — rather than leaning on jargon, and rather than skipping the explanation because the audience is non-technical. Both failure modes appear as distractors: over-technical, and over-simplified to the point of hiding the decision.

AUDIENCE	CARES ABOUT	LEAD WITH
VP Engineering	Delivery risk, SLA, maintainability	The trade-off in latency/cost/quality terms, the recommendation, and what you'd revisit.
Finance	Unit economics	Cost per resolved task, projected volume, assumptions labelled as assumptions.
Legal / compliance	Obligation and evidence	Data classes, regime mapping, controls, audit trail, named accountable owner.
Security	Exposure	Data flows, privilege model, injection posture, blast radius.
Operations	Day-two reality	Runbook, degraded modes, alarms, escalation path.
End users	Their work tomorrow	Concrete before/after, what they still own, honest limitations.

OVERSELLING IS THE TESTED FAILURE

Claiming a system "eliminates errors," "removes the need for review," or "requires no oversight" is both false and consistently the wrong option. The professional framing: **Claude accelerates work that a competent human still reviews and remains accountable for** — and stating the limitations before someone discovers them is what preserves your credibility for the next decision.

6.3 Delivery Lifecycle

~60 MIN

Objectives — Move a system from pilot to production to scale with the right gate at each stage.

STAGE	PURPOSE	EXIT CRITERIA
Discovery	Understand the problem and constraints.	Documented outcome, constraints, risks, and success measures.
Prototype	Test viability cheaply.	Evidence the model can do the task at all.
Pilot	Real users, bounded scope.	Eval set passing, cost and latency measured at real volume, human review in place.
Production	Full rollout with controls.	Governance sign-off, runbook, alarms, rollback path, named owner.
Scale & operate	Grow and maintain.	Continuous evaluation, cost tracking, periodic control review.
Retire / replace	Decommission cleanly.	Data disposition, downstream migration, documented rationale.

THE PILOT'S REAL JOB

A pilot exists to discover **whether the value is real and what production will actually cost** — not to prove the demo works. A pilot with no measurement plan is a demo with users attached, and it produces enthusiasm rather than evidence. Define, before it starts, what result would cause you to stop.

6.4 Operations & Continuous Improvement

~60 MIN

Objectives — Own the system after launch.

- 1 **Monitor the Domain 4 metrics continuously** — accuracy sampling, latency, cost per resolved task, safety and security signals.
- 2 **Alarm on leading indicators** — rising rejection rate, rising iteration counts, falling cache hit rate, shifting stop-reason mix.
- 3 **Feed incidents back** into the eval set permanently.
- 4 **Review controls on a schedule** — permissions and allowlists only ever grow unless someone prunes them.
- 5 **Re-validate on platform change** — new model, new prompt, new tool means re-run the evals.
- 6 **Report to stakeholders in their terms** — outcome and unit economics, not token counts.

DESIGN DRILL 6 — THE STAKEHOLDER BRIEFING

You must recommend a latency-versus-cost trade-off for the report-generation assistant to a **non-technical VP of Engineering**. Write one page: the decision, the two options in user-experience and budget terms, the recommendation, the risk you accepted, and the signal that would make you revisit. **Constraint: no jargon.** No token counts, no model names, no "p95" without explanation. Then read it aloud and cut anything the VP wouldn't act on.

Domain 6 — scenario items

ITEM P6-1 · MODULE 6.1

An e-commerce team began designing a returns-processing assistant without structured discovery with the VP of Customer Experience, and later found the design missed a hard operational constraint. What does this illustrate?

- A. That the VP should have volunteered the constraint earlier
- B. That structured discovery before design exists precisely to surface constraints, outcomes, and risks — skipping it is how hard constraints get missed until rework is expensive
- C. That the design should have been more flexible
- D. That the constraint should be treated as a change request

B. This is a process failure with a known remedy, and naming it as such is the architect's responsibility. A shifts accountability to a stakeholder who was never asked. C treats a discovery gap as an architecture gap — no amount of flexibility substitutes for knowing the constraint. D reframes a missed requirement as scope creep, which protects the schedule and damages the outcome.

ITEM P6-2 · MODULE 6.2

A team must explain a latency-versus-cost trade-off for a report-generation assistant to a non-technical VP of Engineering. Most effective approach?

- A. Present token-level cost analysis and p95 latency distributions
- B. Translate the trade-off into user experience, budget, and SLA impact, with a clear recommendation
- C. Ask the VP to choose between the technical options without a recommendation
- D. Choose internally and report the decision as settled

B. Effective architecture communication meets the audience in the terms they own and still makes a recommendation. A is accurate and unusable for this audience. C abdicates the architect's judgment — presenting options without a recommendation is not neutrality, it's transferring a technical decision to someone unequipped to make it. D removes the stakeholder from a decision that affects their budget and SLA.

ITEM P6-3 · MODULE 6.3

A successful three-week pilot is ready to go to production. Which gap should block the promotion?

- A. The pilot didn't test every possible input
- B. No runbook, no alarms, no rollback path, and no named owner for the production system
- C. The pilot used a different model than planned for production
- D. Users requested additional features

B. These are the operational prerequisites for running a system, and their absence is what turns a good pilot into an unsupportable production service. C is a genuine issue — it would require re-running evaluation (Domain 4) — but is a re-validation step rather than a launch blocker in itself. A is never achievable. D is backlog, not a gate.

ITEM P6-4 · MODULE 6.2

Preparing an executive proposal for a Claude-based assistant. Which framing is most appropriate?

- A. Emphasize headcount reduction from automating the work
- B. Present expected outcomes, the review process that maintains quality, and the limitations requiring human oversight
- C. Emphasize that competitors are adopting AI and the risk of falling behind
- D. Present it as low-risk because outputs are always reviewed

B. Balanced, honest, and it names the control that makes the benefit real. A overpromises a specific organizational outcome. C is pressure rather than a case. **D is the subtle one** — "always reviewed" is usually an aspiration rather than a measured fact, and calling a system low-risk on the strength of a control you haven't yet proven operates is how governance failures begin.

D7 Developer Productivity & Operational Enablement

7% · 2 modules

About 4 of 63 items — the smallest domain, and straightforward points if you've worked in a team that standardized its tooling.

7.1 Standardizing Claude Code Across a Team

~60 MIN

Objectives — Make team tooling consistent, secure, and reviewable.

THE PRINCIPLE

Configuring Claude Code and the development environment **consistently across engineers** supports secure, standardized use. Leaving it to individual preference undermines both consistency and security — behavior varies per machine, bugs stop being reproducible, and no constraint can be relied upon because no one knows whose setup has it.

LAYER	NATURE	BELONGS HERE
Org policy · permission rules · hooks	Enforced	Constraints that must hold regardless of prompt or user. Anything phrased "must never."
Project memory (checked in)	Advisory	Conventions, commands, architecture notes, domain vocabulary, explicit non-goals.
Skills	Advisory	Recurring procedures loaded on demand when relevant.
Slash commands	Advisory	Prompts the team retypes often.
Personal settings	Individual	Ergonomics that don't affect output or security.

CONFIGURATION IS CODE

Shared configuration belongs in version control and goes through review. A hook is executable logic on every engineer's machine; an over-broad permission rule silently widens what runs unattended across the whole team. Review permission lists on a schedule — an allowlist nobody prunes eventually approximates "allow everything."

7.2 AI-Assisted Delivery Workflows

~60 MIN

Objectives — Identify repeatable engineering activities worth accelerating, and the controls that keep them safe.

Applying Claude to a **repeatable part of the software delivery process** — code review, test generation, documentation — is the canonical example of an AI-assisted development workflow that improves productivity. The architect's contribution is choosing which activities qualify and what verification each requires.

ACTIVITY	WHY IT FITS	REQUIRED CONTROL
Code review assistance	High volume, pattern-rich, reviewer stays accountable.	Human approves the merge; treat findings as candidates, not verdicts.
Test generation	Repetitive and immediately verifiable.	Confirm tests fail against broken code — tests that always pass are worse than none.
Documentation	Language-heavy, low reversibility risk.	Technical accuracy review; docs drift from code.
Migration & refactors	Mechanical, large-scale, well-specified.	Tests, staged rollout, reviewable diffs.
Incident summarization	Synthesis across noisy sources.	Verify claims against the underlying evidence.

MEASURE ENABLEMENT LIKE ANY OTHER INVESTMENT

Track cycle time, review turnaround, defect escape rate, and adoption. "Engineers feel faster" is not a result — and unmeasured enablement programs are the first thing cut when budgets tighten, precisely because nobody can show what they returned.

Domain 7 — scenario items

ITEM P7-1 · MODULE 7.1

A platform reliability team wants every engineer working on a report-generation assistant to use a consistent, shared Claude Code configuration rather than each configuring it individually. What's the appropriate approach?

- A. Publish a recommended setup and let engineers adopt it if they prefer
- B. Standardize the configuration in version control, applying shared context and constraints across the team while leaving genuine personal preferences individual
- C. Have each engineer tune their own setup for their workflow
- D. Restrict Claude Code to a single designated engineer

B. Consistent configuration supports secure, standardized use and makes behavior reproducible; leaving it to preference undermines both. A is B without the enforcement, so drift returns. C is the status quo being corrected. D destroys the productivity benefit to solve a governance problem that configuration management already solves.

ITEM P7-2 · MODULE 7.2

A platform team uses Claude to improve a repeatable part of software delivery — code review, test generation, documentation. What is this an example of?

- A. An autonomous agent replacing engineering review
- B. An AI-assisted development workflow that improves delivery productivity, with engineers still accountable for the output
- C. A governance control
- D. A model evaluation technique

B. Repeatable engineering activities are the canonical target for AI-assisted workflows, and the accountability clause is what makes the framing correct. A overstates it — assistance is not replacement, and the merge decision stays human. C and D name different things entirely; the distinction matters because enablement, governance, and evaluation have different owners and metrics.

6-week study plan

Weighted to the blueprint. Integration and Solution Design together are 36% of the exam — they get the most time. Domain 7 is 7% and gets an afternoon.

WEEK	MODULES	WEIGHT	FOCUS
1	1.1 – 1.5	17%	Pattern selection and NFRs. Write ADRs for all four drill scenarios — this is the reasoning every later domain builds on.
2	3.1 – 3.6	19%	Integration, the heaviest domain. Live in the least-privilege material until "remove the capability" is your reflex.
3	4.1 – 4.4 · 2.1 – 2.4	29%	Evaluation strategy and model/context decisions. Build one complete evaluation strategy end to end.
4	5.1 – 5.4	14%	Governance and risk. Memorize the documented limitations and the RAG injection answer.
5	6.1 – 6.4 · 7.1 – 7.2	21%	Stakeholder and lifecycle — the domain engineers underestimate. Do the no-jargon briefing drill out loud.
6	Full review	—	Re-do every scenario item, then run the four simulators under timed conditions.

TIMING DISCIPLINE — 63 ITEMS IN 120 MINUTES

That's about 1:54 per item, and CCAR-P stems are long vignettes. **Read the last sentence of the stem first** — it usually contains the actual question — then read the scenario for the constraint. Flag and move on rather than stalling; multiple-response items state how many to select, so count them. There is no penalty for guessing, so never leave a blank.

HIGHEST-YIELD MATERIAL IF YOU'RE SHORT ON TIME

(1) Least privilege — remove capability, don't monitor it (D3, heaviest domain) · (2) Workflow vs. agent, both directions (D1) · (3) Complete evaluation = accuracy + latency + cost + safety + security, and datasets need edge and ambiguous cases (D4) · (4) RAG injection = treat retrieved content as data + restrict what it can trigger (D5) · (5) Hallucination is a documented limitation to design around, never configured away (D5) · (6) Discovery precedes design; translate trade-offs into the audience's terms (D6).

Those six cover a substantial majority of the item bank's reasoning patterns.

Module tracker

Tick a module only when you can answer its question aloud, without notes, in under a minute.

✓	MOD	TITLE	SELF-ASSESSMENT QUESTION
☐	1.1	Requirement → Architecture	Which requirement words rule out an agent architecture?
☐	1.2	Pattern Selection	What single property decides workflow vs. agent?
☐	1.3	Reference Architecture	Name the seven layers. Which is most often missing?
☐	1.4	Non-Functional Requirements	Which NFR most often invalidates a proposed pattern outright?
☐	1.5	Trade-offs & ADRs	Why can't you optimize latency, cost, and accuracy together?
☐	2.1	Model Selection	Why are "always biggest" and "always cheapest" both wrong?
☐	2.2	Where Instructions Live	What goes in the system prompt vs. an enforced layer?
☐	2.3	Context Engineering	Which mechanism is the only cross-session one?
☐	2.4	Prompt Lifecycle	What three artifacts must change together?
☐	3.1	Integration Topology	What signal means batch rather than synchronous?
☐	3.2	RAG Architecture	State the two RAG rules. What happens with no not-found path?
☐	3.3	Tools & MCP	State the N×M argument. When do you promote an action to its own tool?
☐	3.4	Least Privilege	Why is logging not a substitute for removing a capability?
☐	3.5	Data Boundaries	Name the five steps from classification to audit trail.
☐	3.6	Integration Resilience	Why must a refund tool be idempotent?
☐	4.1	Evaluation Strategy	Name the five dimensions. Why measure per resolved task?
☐	4.2	Eval Datasets	What must a dataset include beyond typical cases, and why?
☐	4.3	Lifecycle Testing	Why is a model swap a change event?
☐	4.4	Optimization	Name the cost levers in order. What must follow every one?
☐	5.1	Documented Limitations	Name four. Why is "eliminated by configuration" always wrong?
☐	5.2	Guardrails & Injection	State the RAG injection answer in one sentence.
☐	5.3	Privacy & Compliance	Why is an eval set built from production traffic a governance event?
☐	5.4	Human Oversight	Name the four requirements for meaningful oversight.
☐	6.1	Structured Discovery	What does skipping discovery reliably cause?
☐	6.2	Communicating Trade-offs	What are the two opposite communication failure modes?
☐	6.3	Delivery Lifecycle	What is a pilot actually for?
☐	6.4	Operations	Name four leading indicators worth alarming on.
☐	7.1	Claude Code Standardization	What does leaving configuration to preference undermine?
☐	7.2	AI-Assisted Delivery	Name three qualifying activities and the control each needs.

ANSWER KEY — QUICK REFERENCE

D1: 1-B · 2-B · 3-B · 4-B · **D2:** 1-B · 2-B · 3-B · **D3:** 1-C · 2-B · 3-B · 4-B · 5-B · **D4:** 1-B · 2-B · 3-B · 4-B ·
D5: 1-B · 2-A · 3-C · 4-B · **D6:** 1-B · 2-B · 3-B · 4-B · **D7:** 1-B · 2-B

The letters cluster because correct professional answers share a shape: name the constraint, prevent rather than observe, keep an accountable human on consequential decisions, and translate for the audience. **Rehearse the reasoning, not the letter** — the real exam shuffles options, and only the rationale transfers.